

Numerical Methods for Quants

From PDEs to Monte Carlo: The Desk-Ready Computational Playbook

Amit Kumar Jha, Quant Analyst at UBS

July 11, 2026

Contents

How to Use This Book	2
Module 1: Root Finding & Optimization (The Calibration Engine)	5
Module 2: Interpolation & Curve Construction (Building the Surface)	15
Module 3: Finite Difference Methods (Solving PDEs on a Grid)	23
Module 4: Monte Carlo Methods (The Universal Pricer)	32
Module 5: Trees & Lattice Methods (The Visual Pricer)	41
Module 6: Fourier & Spectral Methods (The Calibration Accelerator)	48
Module 7: Calibration & Model Selection (Fitting Models to Markets)	55
Module 8: Convergence, Stability & Performance (Production Numerics)	62
Module 9: Finite Element Methods (FEM) for Quants	71
Module 10: Machine Learning in Numerical Finance	78
Interview Toolkit: 15 Classic Numerical Methods Questions	87
Mini Exercises: 15 Scenarios with Solutions	91
How This Note Connects to the Other Playbooks	93

How to Use This Book

First Pass (Foundation): Read *Root Finding + Interpolation + Finite Differences*. Implement 2–3 methods from scratch. Goal: internalize discretization and convergence. Time: 1–2 weeks.

Second Pass (Core Methods): Attack *Monte Carlo + PDE Solvers*. These are the two pillars of quant pricing. Implement Black-Scholes via both FDM and MC. Time: 2–3 weeks.

Third Pass (Production): Read *Calibration + Trees & Lattices + Performance*. Understand when to use which method and why. Time: 1–2 weeks.

Before Interview (48 hours):

- Reread all “Interview Toolkit” sections
- Memorize the method selection decision tree
- Be ready to code Euler–Maruyama, bisection, and Thomas algorithm on a whiteboard
- Prepare your “go-to” stories: one convergence debugging story, one calibration story, one MC variance reduction story

This is not a textbook to read once. It's a computational field manual to annotate and revisit.

Pro Tips from the Desk

1. **“Speed vs Accuracy”:** Every method is a tradeoff. Interviewers want to hear: “For real-time risk, I use [fast method]. For overnight pricing, I use [accurate method].”
2. **“Convergence Order Matters”:** Always state the convergence order. “Euler is $O(\Delta t)$, Milstein is $O(\Delta t)$ strong but $O(\Delta t^2)$ weak” shows depth.
3. **“Stability Before Accuracy”:** A stable wrong answer is better than an unstable right answer. If your PDE solver oscillates, nothing else matters.
4. **“Know Your Grid”:** FDM grid design is 80% of the battle. Log-spot transform, non-uniform mesh near barriers—these details separate juniors from seniors.
5. **“Variance Reduction Every Time”:** Never run a Monte Carlo without at least antithetic variates. It's free accuracy.

Module 2: Interpolation & Curve Construction (Building the Surface)

► Module Overview

What this module is about: Every quant model sits on top of interpolated curves and surfaces: yield curves, vol surfaces, forward curves. If your interpolation is wrong, every price downstream is wrong.

Why this matters: A cubic spline that oscillates between data points will produce negative forward rates or butterfly arbitrage. The market will punish you before your risk system even notices.

What confusion this clears:

- Why linear interpolation creates kinks in Greeks
- Why cubic splines can produce negative forwards
- When to use monotone convex vs natural cubic splines

After this module, you should be able to answer: “Your yield curve has negative forward rates between 2Y and 3Y. What’s wrong and how do you fix it?”

2.1 Linear Interpolation: Simple but Dangerous

Σ Core Formula

$$f(x) = f(x_i) + \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i}(x - x_i), \quad x \in [x_i, x_{i+1}]$$

Properties:

- Continuous but not differentiable at knots.
- Greeks have jumps at every data point.
- No oscillation (monotonicity preserved).

2.2 Cubic Spline Interpolation: Smooth but Risky

Natural Cubic Spline: $S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$ with $S''(x_0) = S''(x_n) = 0$.

Properties:

- C^2 continuous (smooth first and second derivatives).
- Greeks are smooth—no jumps.
- **Risk:** Can oscillate between data points (Runge’s phenomenon).

$\propto$ Proportional Scaling: Monte Carlo Standard Error

The standard error (numerical uncertainty) of a Monte Carlo option price is directly proportional to the asset volatility σ and inversely proportional to the square root of the path count N :

$$\text{SE} \propto \frac{\sigma}{\sqrt{N}}$$

This means that to cut your pricing noise in half, you must run exactly 4 times as many simulation paths.

4.2 The Heston QE (Quadratic-Exponential) Scheme

The Heston variance SDE $dv_t = \kappa(\theta - v_t)dt + \xi\sqrt{v_t}dW_t$ cannot be simulated using standard Euler-Maruyama because v_t can easily become negative. The QE scheme by Leif Andersen uses two different approximations depending on the level of variance v_d :

Given $v_d = v_te^{-\kappa\Delta t} + \theta(1 - e^{-\kappa\Delta t})$, and $s^2 = \frac{v_t\xi^2e^{-\kappa\Delta t}}{\kappa}(1 - e^{-\kappa\Delta t}) + \frac{\theta\xi^2}{2\kappa}(1 - e^{-\kappa\Delta t})^2$. Define $\psi = s^2/v_d^2$.

1. **Case 1: $\psi \leq \psi_c \approx 1.5$ (Low relative variance)** We model the distribution of $v_{t+\Delta t}$ as a non-central chi-square approximated by a power of a normal variable:

$$v_{t+\Delta t} = a(b + Z)^2$$

where $b^2 = 2\psi^{-1} - 1 + \sqrt{2\psi^{-1} - 1}$, $a = \frac{v_d}{1+b^2}$, and $Z \sim N(0, 1)$.

2. **Case 2: $\psi > \psi_c$ (High relative variance)** We model $v_{t+\Delta t}$ as a mixture of a Dirac delta at zero and an exponential distribution:

$$v_{t+\Delta t} = \begin{cases} 0 & \text{if } U < p \\ \frac{1}{\beta} \ln\left(\frac{1-p}{1-U}\right) & \text{if } U \geq p \end{cases}$$

where $p = \frac{\psi-1}{\psi+1}$, $\beta = \frac{1-p}{v_d}$, and $U \sim U[0, 1]$.

```
1 def heston_qe_step(v_t, kappa, theta, xi, dt, psi_c=1.5):
2     # Setup conditional mean and variance parameters
3     e_kt = np.exp(-kappa * dt)
4     v_d = v_t * e_kt + theta * (1 - e_kt)
5     s2 = (v_t * xi**2 * e_kt / kappa) * (1 - e_kt) + (theta * xi**2 / (2*kappa)
6         ) * (1 - e_kt)**2
7     psi = s2 / (v_d**2)
8
9     if psi <= psi_c:
10         # Quadratic approximation
11         b2 = 2/psi - 1 + np.sqrt(2/psi) * np.sqrt(2/psi - 1)
12         a = v_d / (1 + b2)
13         Z = np.random.normal()
14         return a * (np.sqrt(b2) + Z)**2
```

★ Practitioner Insight

Local Volatility Overfitting and SLV Stabilization:

Dupire local volatility matches today's European call option prices exactly. - **Problem:** By forcing the model to fit every wiggle in the option price grid, the local volatility function $\sigma_{\text{loc}}(S, t)$ can exhibit extreme spikes or regions of near-zero volatility. When pricing path-dependent exotics, these spikes create unstable hedging Greek calculations. - **Desk Fix:** Smooth the market implied volatility surface first (using SVI parameterization or spline smoothing under penalization) before running Dupire's formula. When using Stochastic Local Volatility (SLV), verify the leverage function $L(S, t)$ bounds are constrained so that the stochastic variance does not drift into unstable regimes.

★ Memory Anchor: The Model Selection Continuum

- **Local Volatility (Dupire):** Static calibration. Fits the smile perfectly today, but dynamic behavior is wrong (predicts smile flattens over time). Use for vanillas and short-lived options. - **Stochastic Volatility (Heston):** Dynamic calibration. Fits parameters to capture structural smile skew over time, but cannot match every single strike perfectly. Use for long-lived options where volatility dynamics dominate. - **Stochastic Local Volatility (SLV):** Combined calibration. The leverage function restores exact fit to European options while retaining realistic stochastic volatility dynamics. The gold standard for barrier options.

7.5 Novel Quant Trick: Soft Density Penalty for SVI

★ Novel Quant Trick: Soft Density Penalty for SVI

During SVI volatility smile calibration, the parameters must be constrained to prevent butterfly arbitrage. This requires checking that the implied density $g(k) \geq 0$ across all log-strikes k . Enforcing this as a hard boundary constraint makes the optimization surface highly discontinuous, causing gradient solvers (like L-BFGS-B) to fail or stall near the boundary.

The Trick: Rather than using hard constraints, incorporate a **soft density penalty** directly into the optimization loss function. 1. Define a target minimum density threshold $\epsilon = 10^{-4}$. 2. In the calibration loss, add a penalty term proportional to the violation:

$$\text{Loss}_{\text{total}}(\Theta) = \sum_i (\sigma_{\text{model}}(k_i; \Theta) - \sigma_{\text{mkt}}(k_i))^2 + \lambda \sum_j \max(0, \epsilon - g(k_j; \Theta))^2$$

where k_j is a fine grid of log-strikes covering the wings (e.g., $k \in [-4, 4]$) and $\lambda \approx 10^5$ is the penalty weight. 3. This creates a smooth "soft barrier" that gently pushes the optimizer back into the arbitrage-free zone, allowing local gradient-based solvers to converge rapidly without requiring complex non-linear constraint algorithms.